

3. Data Processing

“Without a systematic way to start and keep data clean, bad data will happen.”

— Donato Diorio

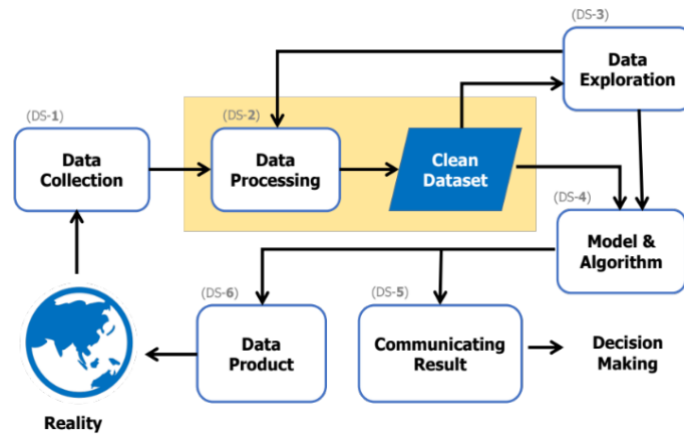


Figure 3.0-1: Data processing step in the data science process

If the analytics process were compared to cooking, data collection would be like gathering ingredients from the garden. Data processing (Figure 3.0-1, DS-2) is like preparing those ingredients before cooking — washing, cutting, and portioning them, just as data must be made clean and ready (a Clean Dataset) before moving on to the cooking step itself: building the model.

3.1 Why Data Must Be Processed

In practice, data rarely arrives in the exact form we need, since collection methods, data structures, tools, units, and recording practices all vary — not to mention human or equipment error. Raw data therefore usually cannot go straight into model building; it typically has missing values or abnormal values that must be cleaned and transformed into a ready-to-use form first (O’Neil & Schutt, 2013).

3.2 Approaches to Cleaning Data

“Dirty” data usually stems from two problems: missing values and outliers. Missing values tend to break computer processing outright (causing errors that halt the analysis), while outliers don’t usually break processing but reduce model accuracy (Igual & Segui, 2017). The following uses Table 3.2-1 as a running example.

Table 3.2-1: Example of data that is not yet ready for use

name	gender	height	weight	score
Anda	female	150		30
Bordin	male	170	63	30
Chantana	female	145	45	35
Donlaya	female	155	50	35
Ekkasit	male	178	75	30
Fundee	female	152	51	40
Gitiwit		175	68	25
Harit	male	172	80	190

Handling Missing Values

In Table 3.2-1, Anda's weight and Gitiwit's gender are missing. Common approaches:

Deleting Rows

Simply drop the entire row (e.g., Anda's), which is the simplest option but reduces the dataset size. This is appropriate only if the remaining data is still large enough for analysis; a threshold rule (e.g., drop a row only if it has more than a certain number of missing fields) can also be used.

Imputing Values

To avoid losing data, missing cells can be filled with a constant or a computed value:

- Default value — for numeric data, a value such as 0 may seem convenient, but a weight of 0 is unrealistic, so this is not recommended; for nominal data, a placeholder such as “Unknown” effectively creates a new, unrealistic category, so this is also discouraged.
- Expert judgment — a domain expert fills in the value, useful mainly for nominal data (e.g., inferring that “Gitiwit” is likely a male name), though this doesn't scale well to large datasets.
- Central-tendency imputation — a popular approach: use the mean for numeric columns (e.g., Anda's missing weight becomes the average of all weights, 61.7) or the mode for nominal columns (e.g., the most common gender, Female, for Gitiwit). This is statistically reasonable but can still look unrealistic in specific cases (a weight of 61 assigned to a smaller-framed row, or Gitiwit being labeled Female).
- Group-wise central-tendency imputation — a smarter variant: impute using the mean within a relevant subgroup, e.g., the average weight among females (48.7) for Anda's missing weight — more realistic than using the overall table average.
- Machine-learning-based imputation — predict the missing value using other columns, e.g., predicting weight from height via Regression Analysis, or predicting a missing nominal value like gender via Classification Analysis. This requires more advanced modeling knowledge but tends to be the most realistic.

Handling Outliers

An outlier is a value that falls well outside the normal range — it may be genuine or the result of a data-collection error. For example, a room-temperature sensor reporting every minute normally reads 24–28°C, but occasionally malfunctions and reports 0 or 255. Even when genuine, such extreme values can distort central-tendency statistics like the mean, so analysts must identify and handle them using approaches such as the following.

Detecting Outliers via Percentile

A simple approach: compute percentiles for the column and treat a chosen range — e.g., [1, 99], [5, 95], or [10, 90] — as “normal.”

For the score column in Table 3.2-1 — [30, 30, 35, 35, 30, 40, 25, 190] — using the [5, 95] percentile range, the 5th percentile is 26.75 and the 95th percentile is 137.5, so both 25 and 190 would be flagged as outliers.

Here, 190 is reasonably flagged as an outlier since it's far from the rest of the group, but 25 is still close to the cluster and probably shouldn't be. Because this method ignores how data actually clusters, other techniques based on the data's distribution are often preferred.

Detecting Outliers via Z-Score

The z-score uses the normal distribution, assuming most values cluster near the center with few far away, and scores each value as follows:

$$z = \frac{x - \mu}{SD}$$

where z is the standard score, x is a given value, μ is the mean, and SD is the standard deviation (covered in the next chapter). Once scores are computed, they fall along a normal curve (Figure 3.2-1). In practice, analysts pick an acceptable range — e.g., $[-2, 2]$ (about 95% of the data) or $[-3, 3]$ (about 99.7%) — and flag anything outside that range as an outlier.

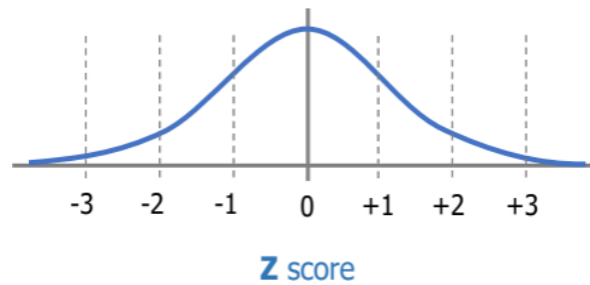


Figure 3.2-1: Standard scores on the normal curve

For the score column [30, 30, 35, 35, 30, 40, 25, 190], the mean is 51.9 and the standard deviation is 56, giving z-scores of [-0.39, -0.39, -0.30, -0.30, -0.39, -0.21, -0.48, 2.47]. Using an acceptable range of [-2, 2], the score of 190 ($z = 2.47$) falls outside the range and is flagged as an outlier.

Detecting Outliers via IQR

The Interquartile Range (IQR) method computes:

$$IQR = Q3 - Q1$$

$$Lower\ bound = Q1 - 1.5(IQR)$$

$$Upper\ bound = Q3 + 1.5(IQR)$$

Any value below the lower bound or above the upper bound is an outlier. For the score column [30, 30, 35, 35, 30, 40, 25, 190], $Q1 = 30$ and $Q3 = 36.25$, so:

$$IQR = 36.25 - 30 = 6.25$$

$$Lower\ bound = 30 - 1.5(6.25) = 20.63$$

$$Upper\ bound = 36.25 + 1.5(6.25) = 45.63$$

Only 190 falls outside this range (above the upper bound), so it is flagged as the outlier.

Handling Outliers

Once identified, outliers can be handled in two ways:

- Drop — remove the entire row containing the outlier; consider whether this discards other valuable information first.
- Cap — clip the value to the nearest bound. For example, with an IQR upper bound of 45.63, the outlier value 190 would be replaced with 45.63.

3.3 Approaches to Transforming Data

Data transformation is a form of feature engineering that reshapes data to suit processing needs. Seven techniques are covered below.

Logarithm Transformation

If plotted data isn't linear, applying a logarithm (log) function may straighten it out, making it more suitable for linear regression, as shown in Figure 3.3-1: the left plot shows the raw X, Y pairs; the right plot shows Y transformed to $\log(Y)$, producing a linear pattern.

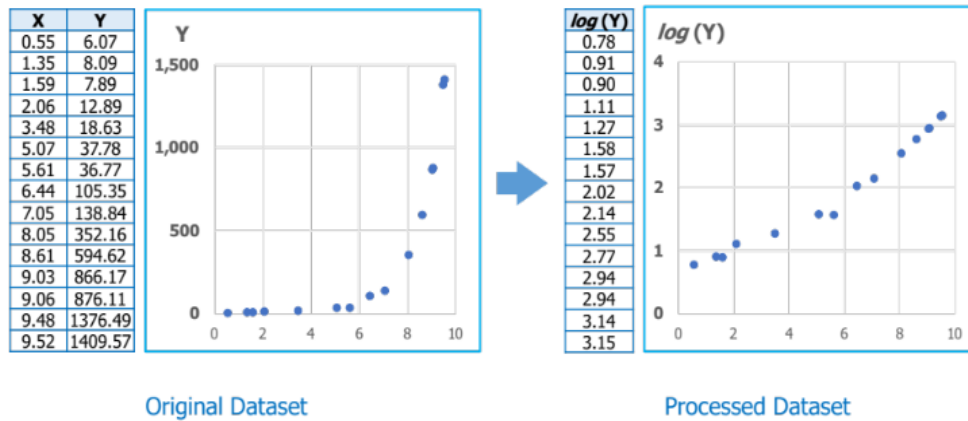


Figure 3.3-1: Example of logarithm transformation

Encoding

Encoding converts nominal data into numbers, since some analytics techniques require numeric input. In Figure 3.3-2, the original Gender column (Male, Female) is converted to 0 and 1 respectively.

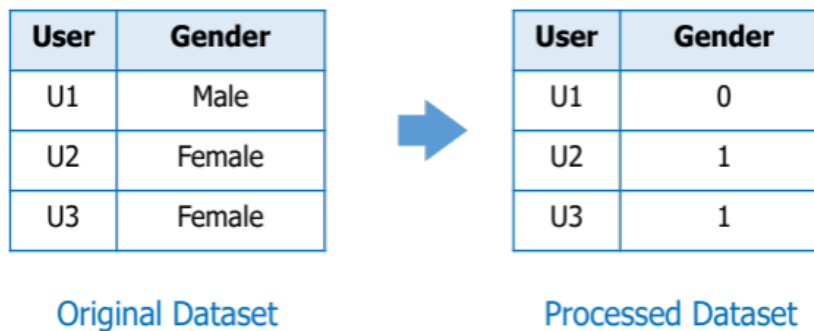


Figure 3.3-2: Example of encoding

For binary categories like Male/Female, any numeric mapping works since order doesn't matter. For ordinal data with 3+ categories, such as clothing sizes XS, S, M, L, XL, sequential encoding (0, 1, 2, 3, 4) is valid, since L being greater than M and less than XL is a meaningful interpretation. However, for nominal data with 3+ unordered categories — e.g., Facebook, Twitter, Instagram — sequential encoding (0, 1, 2) must be avoided, since it would wrongly imply Twitter > Facebook but < Instagram. One-Hot Encoding should be used instead.

One-Hot Encoding

One-Hot Encoding converts a nominal column with 3+ categories (or cells holding multiple values) into separate new columns, each counting occurrences. In Figure 3.3-3, a Social Media column containing multiple values per cell is split into three new columns — Facebook, Twitter, and Instagram — with counts filled in accordingly (e.g., user U1 has 1 in Facebook but 0 in Instagram, since U1 doesn't use Instagram).

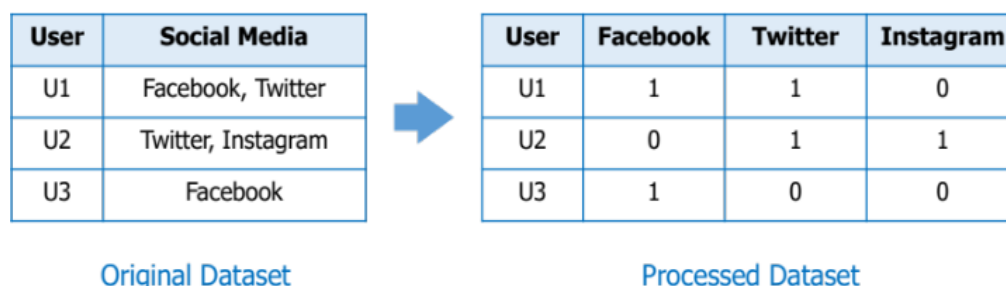


Figure 3.3-3: Example of one-hot encoding

Binning

If continuous numeric data needs to become categorical for a given technique, binning groups values into ranges, as in Figure 3.3-4, where age 15 becomes “Teen” and ages 25 and 30 become “Adult.” This requires either domain expertise or clustering techniques to define sensible groups.

User	Age		User	Age
U1	15	➔	U1	Teen
U2	25		U2	Adult
U3	30		U3	Adult
Original Dataset			Processed Dataset	

Figure 3.3-4: Example of binning

Scaling

Columns are often on very different numeric scales, as in Figure 3.3-5, where Age is in the tens while Salary is in the tens of thousands — a difference that can affect some analysis techniques. Two common scaling approaches address this.

Min-Max Normalization

A simple, direct technique:

$$x' = \frac{x - \min}{\max - \min}$$

where x' is the transformed value, x is the original value, and $\min/\max$ are the column's minimum and maximum. Applying this to both Age and Salary brings both into the same $[0, 1]$ range, as shown on the right side of Figure 3.3-5.

User	Age	Salary		User	Age	Salary
U1	15	30,000	➔	U1	0.0	0.0
U2	25	50,000		U2	0.7	0.4
U3	30	80,000		U3	1.0	1.0
Original Dataset				Processed Dataset		

Figure 3.3-5: Example of min-max normalization

Standardization

Standardization uses the same z-score formula introduced earlier:

$$z = \frac{x - \mu}{SD}$$

where z is the standard score, x is a given value, μ is the mean, and SD is the standard deviation (covered in the next chapter).

Splitting

Sometimes data needs to be split apart for easier processing. In Figure 3.3-6, the left table lists each user's specific phone model; splitting this data (right table) makes it possible to count users by brand — iPhone or Galaxy — regardless of model.

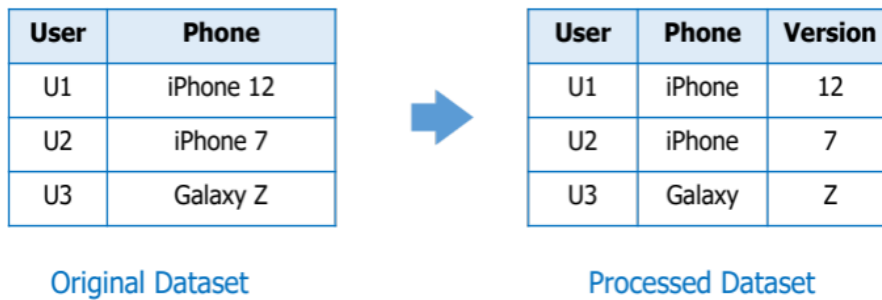


Figure 3.3-6: Example of splitting

Extracting Values from Dates

Dates are commonly stored as “YYYY-MM-dd HH:mm:ss” (year, month, day, hour, minute, second). For example, “2021-06-14 09:45:36” can be broken down into:

- Year: 2021
- Month: 06, or June, or JUN
- Day: 14
- Hour: 09
- Minute: 45
- Second: 36

Further derived features can also be computed, such as:

- Day of week: Monday, or MON
- Whether it's a working day
- Week 24 of the year 2021
- Day 165 of the year 2021
- 200 days remaining until the end of 2021

Such derived values are often useful as features in later analysis.

3.4 Using Python for Data Analytics

Python suits data-analytics programming well, thanks to its rich ecosystem of libraries covering common analytics techniques, sparing developers from implementing methods from scratch. This book demonstrates two popular ways to use Python: Google Colab and Anaconda.

Google Colab

Google Colab is a browser-based, notebook-style environment for writing Python online using Google's cloud compute resources, available at:

<https://colab.research.google.com/>

After signing in with a Google account, users can create a notebook (currently running Python 3), write code, and see results on the next line, as shown in Figure 3.4-1. Colab comes pre-installed with the key data-analytics libraries, so code runs immediately, and results can easily be shared online. This book presents its code examples primarily via Google Colab for consistency.

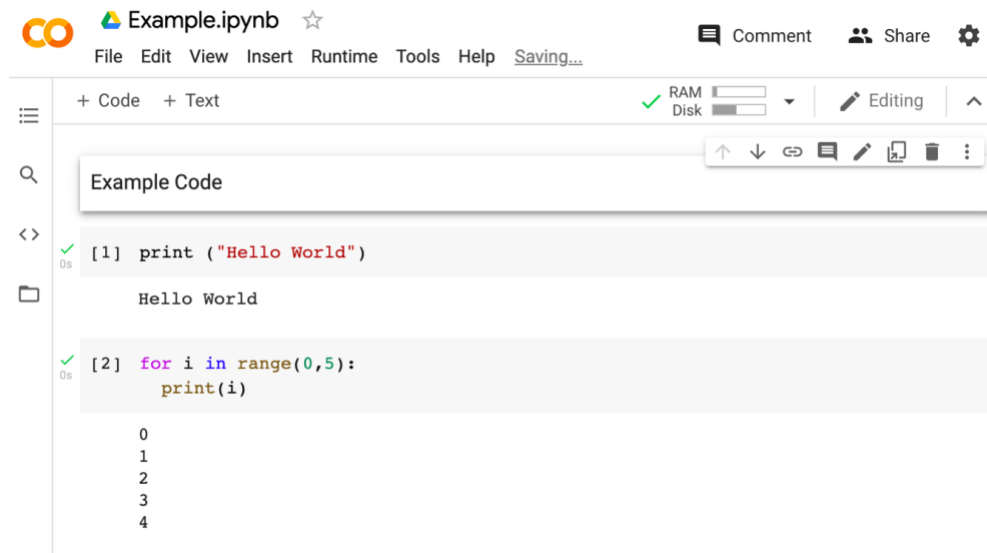


Figure 3.4-1: The Google Colab interface in a web browser

Anaconda

For analytics work using local computer resources — local files, CPU, memory, and storage — Python, the relevant libraries, and a development tool such as PyCharm or Spyder must be installed manually, which can be cumbersome for beginners. Anaconda bundles and installs the necessary tools together, available at:

<https://www.anaconda.com/>

After installation, the Anaconda Navigator (Figure 3.4-2) provides access to bundled tools, including JupyterLab and Jupyter Notebook (browser-based, similar to Colab), and Spyder, which the author recommends for its convenient layout for writing code and viewing results side by side, as shown in Figure 3.4-3: (A) the project file browser, (B) the code editor, (C) the plot viewer, and (D) the console output.

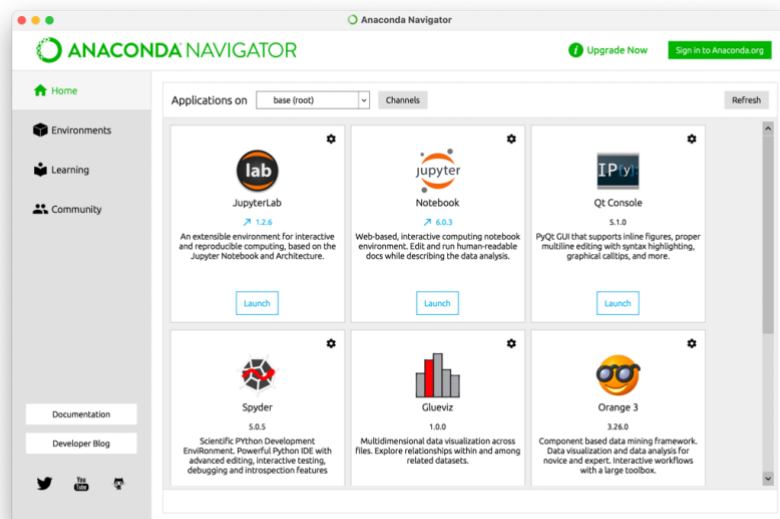


Figure 3.4-2: The Anaconda Navigator interface

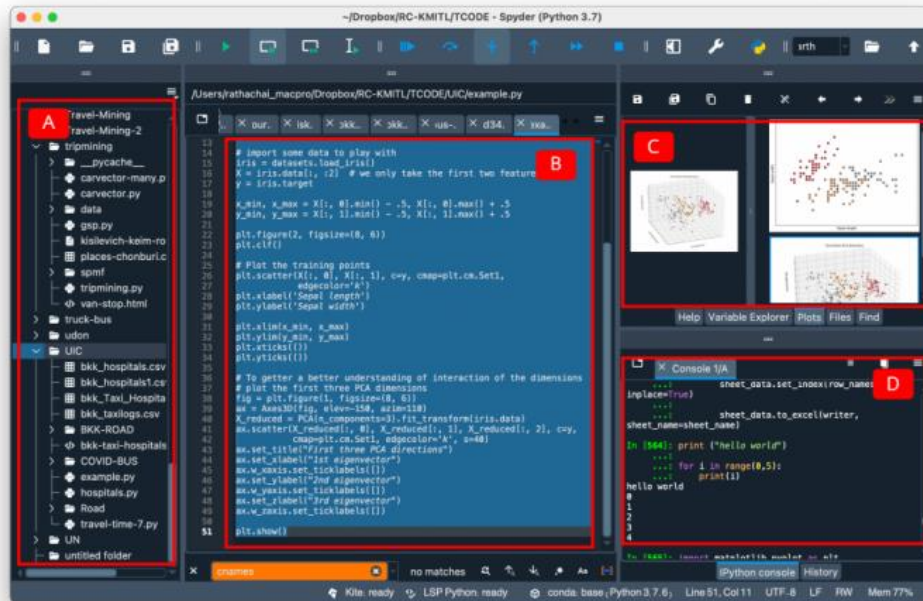


Figure 3.4-3: The Spyder application interface

3.5 Python Data Structures

Python itself can be learned in depth at <https://www.python.org/>; this section only briefly covers what is needed for data-analytics programming going forward — primitive data types, lists, dictionaries, and tuples — not a full language reference.

Primitive Data Types

Four primitive types matter most for analytics work:

1. Integer, e.g., -1, 0, 1, 2
2. Floating point, e.g., 3.14
3. Boolean: True or False
4. String, e.g., "hello world"

```
int_var = 100
float_var = 100.111
bool_var = True
string_var = "hello world"
```

List

A list is an array-like, ordered structure, accessible by index and mutable:

```
>>> my_list = [0, 10, 20]
>>> my_list[1]
10
>>> my_list[1] = 100
>>> my_list
[0, 100, 20]
```


Dictionary

A dictionary holds key–value pairs, is unordered, mutable, and accessed by key:

```
>>> my_dict = {'b':'blue', 'g':'green', 'r':'red'}
>>> my_dict['g']
'green'
>>> my_dict['g'] = 'GREEN'
>>> my_dict
{'b': 'blue', 'g': 'GREEN', 'r': 'red'}
```

Tuple

A tuple is an ordered array like a list, accessible by index, but immutable:

```
>>> my_tuple = (0, 10, 20)
>>> my_tuple[1]
10
>>> a, b, c = my_tuple
>>> b
10
```

3.6 Using the NumPy Library

Data-analytics code usually works with vectors and matrices via mathematical operations. For example, adding vectors `[1, 2]` and `[10, 20]` should give `[11, 22]`, but Python's built-in list simply concatenates: `[1, 2] + [10, 20]` gives `[1, 2, 10, 20]`. Similarly, `[1, 2]*3` gives `[1, 2, 1, 2, 1, 2]` (repetition), not the expected `[3, 6]`. While a for-loop can work around this, it adds unnecessary complexity.

For this reason, Python analysts rely on NumPy, the standard library for vector and matrix operations with built-in statistical functions. Popular higher-level libraries such as scikit-learn are themselves built on NumPy, so learning NumPy well is essential (see <https://numpy.org/>) (Boschetti & Massaron, 2015; VanderPlas, 2016).

Importing NumPy

Google Colab and Anaconda come with NumPy pre-installed, so it can be imported directly, conventionally under the alias `np`:

```
import numpy as np
```

Creating Arrays

NumPy vectors and matrices use the array structure, created via `np.array` from a list:

```
>>> np.array([1,2,3])
array([1, 2, 3])
>>> a = np.array([1,2,3])
>>> b = np.array([10,20,30])
>>> a+b
array([11, 22, 33])
>>> a*3
array([3, 6, 9])
```

Basic Math and Statistical Functions

NumPy is also popular for its math functions (sin, cos, tan, log, sqrt, etc.) and statistical functions (min, max, sum, count, mean, std, etc.) — see <https://numpy.org/> for the full reference.

```
>>> my_array = np.array([1,2,3,4])
>>> np.sqrt(my_array)
array([1., 1.41421356, 1.73205081, 2.])
>>> np.sum(my_array)
10
>>> np.mean(my_array)
2.5
```

3.7 Using the Pandas Library

Most real-world analytics work involves tabular data, which could be represented as a matrix using NumPy — but matrix indexing by row/column number isn't natural for analysts, who prefer referencing columns by name. This is why the Pandas DataFrame is the standard structure for tabular data; since Pandas is built on NumPy, its usage is consistent with NumPy conventions (Boschetti & Massaron, 2015; VanderPlas, 2016). Analysts should master Pandas' structures, functions, and techniques — see <https://pandas.pydata.org/> for the official documentation.

Importing Pandas

Like NumPy, Pandas comes pre-installed in Colab and Anaconda, and is conventionally imported under the alias `pd`:

```
import pandas as pd
```

Creating a DataFrame

A DataFrame can be created from Python lists, dictionaries, or NumPy arrays via `pd.DataFrame`, or by reading a CSV, Excel, HTML file, or a database query. The example below reads a CSV file into a DataFrame:

```
CSV_PATH = "https://rathachai.github.io/DA-LAB/datasets/simple-employee-db.csv"
df = pd.read_csv(CSV_PATH)
df
```

	eid	name	gender	department	age	salary	working_years	birth_place
0	E011	Anda	female	developer	39	64200	4	Bangkok
1	E012	Bordin	male	developer	25	48700	2	Phuket
2	E013	Chantana	female	developer	29	45500	3	Chonburi
3	E014	Donlaya	female	marketing	39	72600	8	Chonburi
4	E015	Ekkasit	male	marketing	37	80500	7	Suphanburi
5	E016	Fundee	female	support	35	56600	3	Phuket
6	E017	Gitiwit	male	support	26	42400	2	Suphanburi
7	E018	Harit	male	devops	32	67700	5	Bangkok

Selecting Columns and Rows

Columns and rows can be selected by name. Selecting a single column with single brackets returns a Series; double brackets return a single-column DataFrame; a list of column names returns a multi-column DataFrame:

```
>>> df["name"]
0      Anda
1    Bordin
2  Chantana
3   Donlaya
4   Ekkasit
5   Fundee
6   Gitiwit
7     Harit
Name: name, dtype: object

>>> df[["name","gender","age"]]
   name  gender  age
0  Anda  female   39
1  Bordin   male   25
2  Chantana  female   29
3  Donlaya  female   39
4  Ekkasit   male   37
5  Fundee   female   35
6  Gitiwit   male   26
7   Harit   male   32
```

Rows are selected using `loc` with a row index (or a list of indices for a DataFrame result); numeric indices also support slice notation such as `df.loc[1:3]`:

```
>>> df.loc[1]
eid      E012
name      Bordin
gender    male
department developer
age        25
salary    48700
working_years  2
birth_place  Phuket
Name: 1, dtype: object

>>> df.loc[[1,2,3]]
   eid  name  gender  department  age  salary  working_years  birth_place
1  E012  Bordin   male  developer   25   48700             2      Phuket
2  E013  Chantana  female  developer   29   45500             3    Chonburi
3  E014  Donlaya  female  marketing   39   72600             8    Chonburi
```

Rows and columns can also be selected together — simultaneously, columns-then-rows, or rows-then-columns — all giving the same result:

```
>>> df.loc[[1,2,3], ["name","age"]]
>>> df[["name","age"]].loc[[1,2,3]]
>>> df.loc[[1,2,3]][["name","age"]]
   name  age
1  Bordin   25
2  Chantana  29
3  Donlaya  39
```

Rows can also be filtered by condition, using & for logical AND and | for logical OR, with each expression in parentheses:

```
>>> df[(df["working_years"]>3) & (df["salary"]>70000)]
   eid  name gender department  age  salary  working_years  birth_place
3  E014 Donlaya female marketing   39   72600             8     Chonburi
4  E015 Ekkasit   male marketing   37   80500             7     Suphanburi
```

Editing Data

A specific cell can be edited directly via loc, giving the row index and column name:

```
>>> df.loc[0,"working_years"] = 77
>>> df
   eid  name gender department  age  salary  working_years
birth_place
0  E011  Anda  female  developer   39   64200             77
Bangkok
1  E012  Bordin   male  developer   25   48700             2
Phuket
2  E013  Chantana female  developer   29   45500             3
Chonburi
3  E014  Donlaya female marketing   39   72600             8
Chonburi
4  E015  Ekkasit   male marketing   37   80500             7
Suphanburi
5  E016  Fundee  female   support   35   56600             3
Phuket
6  E017  Gitiwit   male   support   26   42400             2
Suphanburi
7  E018  Harit    male    devops   32   67700             5
Bangkok
```

Setting the Row Index

By default, rows are indexed by an auto-incrementing integer starting at 0, but a column with unique values can be set as the index using set_index (with inplace=True to modify the DataFrame directly); reset_index restores the default integer index:

```
>>> df.set_index("eid", inplace=True)
>>> df
   name gender department  age  salary  working_years  birth_place
eid
E011  Anda  female  developer   39   64200             4     Bangkok
E012  Bordin   male  developer   25   48700             2     Phuket
E013  Chantana female  developer   29   45500             3     Chonburi
E014  Donlaya female marketing   39   72600             8     Chonburi
E015  Ekkasit   male marketing   37   80500             7     Suphanburi
E016  Fundee  female   support   35   56600             3     Phuket
E017  Gitiwit   male   support   26   42400             2     Suphanburi
E018  Harit    male    devops   32   67700             5     Bangkok

>>> df.reset_index(inplace=True)
>>> df
   eid  name gender department  age  salary  working_years  birth_place
0  E011  Anda  female  developer   39   64200             4     Bangkok
1  E012  Bordin   male  developer   25   48700             2     Phuket
...
```

Processing Column Values

Each column behaves like a vector and supports arithmetic with a constant or another column:

```
>>> df["salary"]*12
0    770400
1    584400
2    546000
3    871200
4    966000
5    679200
6    508800
7    812400
Name: salary, dtype: int64

>>> df["salary"]*df["working_years"]
0    256800
1     97400
2   136500
3   580800
4   563500
5   169800
6    84800
7   338500
dtype: int64
```

A function can also be applied to every value in a column via `apply`, including NumPy functions or a custom lambda function:

```
>>> df["salary"].apply(np.sqrt)
0    253.377189
1    220.680765
2    213.307290
3    269.443872
4    283.725219
5    237.907545
6    205.912603
7    260.192237
Name: salary, dtype: float64

>>> df["salary"].apply(lambda x: "high" if x>50000 else "low")
0    high
1     low
2     low
3    high
4    high
5    high
6     low
7    high
Name: salary, dtype: object
```

New columns can be added directly, whether with a constant value or a computed one:

```
>>> df["country"] = "Thailand"
>>> df["bonus"] = df["salary"]*df["working_years"]
>>> df
   eid   name  gender  ... birth_place  country  bonus
0  E011   Anda  female  ...   Bangkok  Thailand  256800
1  E012  Bordin   male  ...    Phuket  Thailand   97400
2  E013 Chantana  female  ...   Chonburi  Thailand  136500
3  E014  Donlaya  female  ...   Chonburi  Thailand  580800
4  E015  Ekkasit   male  ...  Suphanburi  Thailand  563500
5  E016  Fundee  female  ...    Phuket  Thailand  169800
6  E017  Gitiwit   male  ...  Suphanburi  Thailand   84800
7  E018   Harit   male  ...   Bangkok  Thailand  338500

[8 rows x 10 columns]
```

Dropping Rows and Columns

Rows or columns are removed with drop, given a list of row/column labels; inplace=True makes the change permanent, and axis=1 indicates columns (axis=0, the default, means rows):

```
>>> df.drop([0,1,2], inplace=True)
>>> df.drop(["country","bonus"], axis=1, inplace=True)
>>> df
   eid   name  gender  department  age  salary  working_years  birth_place
3  E014  Donlaya  female  marketing   39   72600             8    Chonburi
4  E015  Ekkasit   male  marketing   37   80500             7    Suphanburi
5  E016  Fundee  female  support    35   56600             3      Phuket
6  E017  Gitiwit   male  support    26   42400             2    Suphanburi
7  E018   Harit   male  devops     32   67700             5      Bangkok
```

Sorting Data

sort_values sorts by a given column, ascending by default; use ascending=False for descending order:

```
>>> df.sort_values("salary")
   eid   name  gender  department  age  salary  working_years  birth_place
6  E017  Gitiwit   male  support    26   42400             2    Suphanburi
2  E013  Chantana  female  developer   29   45500             3    Chonburi
1  E012  Bordin   male  developer   25   48700             2      Phuket
5  E016  Fundee  female  support    35   56600             3      Phuket
0  E011   Anda  female  developer   39   64200             4      Bangkok
7  E018   Harit   male  devops     32   67700             5      Bangkok
3  E014  Donlaya  female  marketing   39   72600             8    Chonburi
4  E015  Ekkasit   male  marketing   37   80500             7    Suphanburi

>>> df.sort_values("salary", ascending=False)
      (same rows, in reverse order)
```

Group-wise Statistics

groupby groups rows by one or more columns, then applies a statistical function (mean, min, max, count, etc.) to each group. Multiple summary functions can be applied at once via agg, and different functions can be applied to different columns using a dictionary:

```
>>> df.groupby("department").mean()
      age  salary  working_years
department
developer  31.0  52800.0          3.0
devops     32.0  67700.0          5.0
marketing  38.0  76550.0          7.5
support    30.5  49500.0          2.5

>>> df.groupby(["department", "gender"]).mean()
      age  salary  working_years
department gender
developer  female  34.0  54850.0          3.5
           male   25.0  48700.0          2.0
devops     male   32.0  67700.0          5.0
marketing  female  39.0  72600.0          8.0
           male   37.0  80500.0          7.0
support    female  35.0  56600.0          3.0
           male   26.0  42400.0          2.0

>>> df.groupby("department").agg(["mean", "count"])
      age  salary  working_years
      mean count  mean count  mean count
department
developer  31.0     3  52800.0     3     3.0     3
devops     32.0     1  67700.0     1     5.0     1
marketing  38.0     2  76550.0     2     7.5     2
support    30.5     2  49500.0     2     2.5     2

>>> df.groupby("department").agg({"age": ["min", "max"], "salary": "mean"})
      age  salary
      min max  mean
department
developer  25  39  52800.0
devops     32  32  67700.0
marketing  37  39  76550.0
support    26  35  49500.0
```

Saving Data to a File

A processed DataFrame can be saved in several formats, including Excel, CSV, and JSON. The example below saves to CSV via `to_csv`:

```
df.to_csv("employee.csv")
df.to_csv("employee.csv", index=False)
df.to_csv("employee.csv", index=False, encoding='utf-8-sig')
```

By default the row index is written as the first column (appearing blank if it's just a sequential number). Use `index=False` to omit it. If the DataFrame contains Thai or other non-English text, use `encoding='utf-8-sig'` so the file can be read back correctly.

3.8 Chapter Summary

This chapter covered preparing raw data for use: cleaning missing values and outliers, transforming data into features better suited for analysis, and using Python for data processing — including the tools available, basic Python data structures, linear-algebra operations with NumPy, and tabular data handling with Pandas, giving readers a foundation to build on in later chapters.

3.9 Review Questions

Using Table 3.9-1 below, answer the following questions.

Table 3.9-1: Example student data

name	faculty	gpa	hobby	exercise_minutes
Anda	Engineering	3.52	Football	310
Bordin	Science	3.45	Swimming	125
Chantana	Engineering	2.62	Swimming	170
Donlaya	Science	3.10	Swimming	
Ekkasit	Science	3.26	Football	1200

1. Explain how you would handle the missing value(s) in the table.
2. Explain how you would detect and handle any outlier(s) in the table.
3. Transform the hobby column using One-Hot Encoding.
4. Transform the gpa and exercise_minutes columns onto the same scale using Min-Max Normalization.
5. Assuming this table is stored as a Pandas DataFrame df (with numpy imported as np and pandas as pd), answer the following:
 - a. Sort the rows by gpa in descending order.
 - b. Select the name and hobby columns only for students with gpa greater than 3.4.
 - c. Count the number of students per hobby, broken down by faculty.
 - d. Find the minimum gpa within each faculty.
 - e. Find the names of students whose average gpa exceeds 3.4 and whose hobby is Football.

3.10 References

Boschetti, A., & Massaron, L. (2015). Python Data Science Essentials. Packt Publishing Ltd.

Igual, L., & Seguí, S. (2017). Introduction to Data Science. Springer, Cham.

O'Neil, C., & Schutt, R. (2013). Doing Data Science: Straight Talk from the Frontline. O'Reilly Media, Inc.

VanderPlas, J. (2016). Python Data Science Handbook: Essential Tools for Working with Data. O'Reilly Media, Inc.